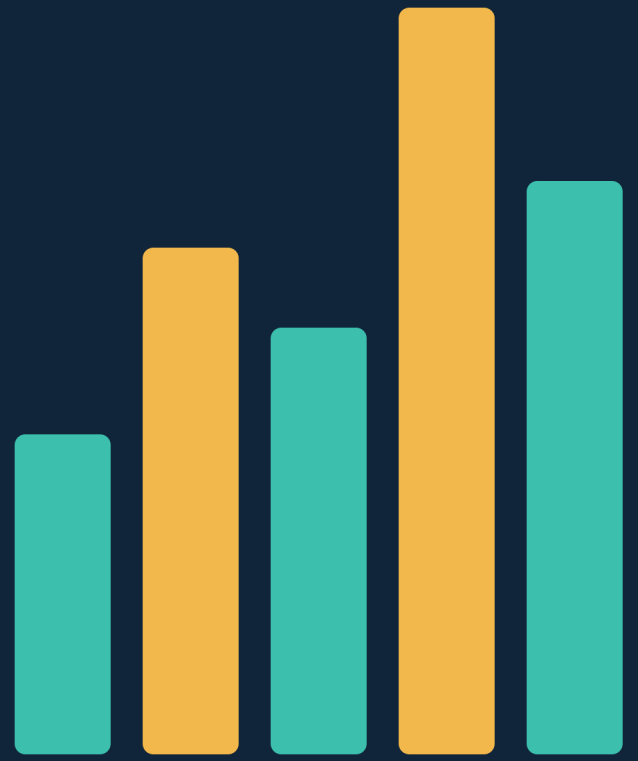
PBL PROJECT 3 · PYTHON

Personal Expense Tracker & Analysis System

Record daily spending, analyse it with Pandas and see it with Matplotlib.



Student name(s): [Student Name(s)]

Roll number(s): [Roll No(s).]

Department: [Department]

Subject: [Subject Name] · Code: [Subject Code]

01 · INTRODUCTION

Background and overview

Knowing where money goes is the first step to managing it. Students and young professionals spend every day on food, travel, study material and entertainment, yet seldom keep a record.

This project is a Python application that records each expense, saves it to a file, and turns the records into totals, averages, monthly summaries and charts.

Record
Date, category, amount, description

Analyse
Pandas totals, averages, extremes

Visualise
Matplotlib bar, pie, line, histogram

Money is spent every day, but there is no simple way to see where it goes.

The need: a lightweight tool that captures each expense correctly and explains spending in numbers and pictures.

01 Records are scattered across notebooks, phones and memory, or never kept at all.

02 Totals worked out by hand are slow, and one wrong entry spoils the result.

03 Without summaries or charts, spending patterns stay invisible.

03 · OBJECTIVES

Aims and expected outcomes

01

Record expenses with date, category, amount and description.

02

Store records permanently in a CSV file and reload them on start.

03

Calculate total, monthly, category-wise and average spending.

04

Identify the highest and lowest expenses and sort the records.

05

Visualise spending with bar, pie, line and histogram charts.

06

Reject invalid amounts and dates so the data stays clean.

Expected outcome: a menu-driven tool that turns daily entries into clear spending insights.

04 · SCOPE OF THE PROJECT

Coverage, users and limitations

Coverage

- Daily expense entry by category
- Total, monthly and category summaries
- Sorting, highest and lowest expenses
- Four chart types
- Saving and loading from a CSV file

Target users

- Students managing a monthly allowance
- Young professionals tracking a personal budget
- Beginners learning Python data analysis

Limitations

- One user, one local CSV file
- Text-menu interface, no graphical window
- No edit or delete for saved records yet
- Single currency (Rs.)
- No automatic bank or UPI import

Common ways to track expenses

Approach	How it works	Limitation
Paper notebook or diary	Expenses written by hand each day	Easy to lose; totals and charts are manual
Spreadsheet (Excel, Sheets)	Rows and formulas set up by the user	Needs set-up; no input checking; charts built by hand
Mobile budgeting apps	Ready-made screens and reports	May need sign-up or internet; data stays with the provider
Bank or UPI statements	Automatic list of payments	No personal categories or notes; cash is missing

Gap: a free, offline, customisable tool that keeps your own data and analyses it automatically.

06 · PROPOSED SYSTEM

Six key features

Add expense

Enter date, category, amount and a note through a guided menu.

Saved records

Every entry is written to expenses.csv and reloaded next time.

Analysis

Total, monthly, category-wise and average spending in one view.

Sort and extremes

Sort by date, amount or category; spot the highest and lowest.

Charts

Bar, pie, line and histogram charts, also saved as PNG files.

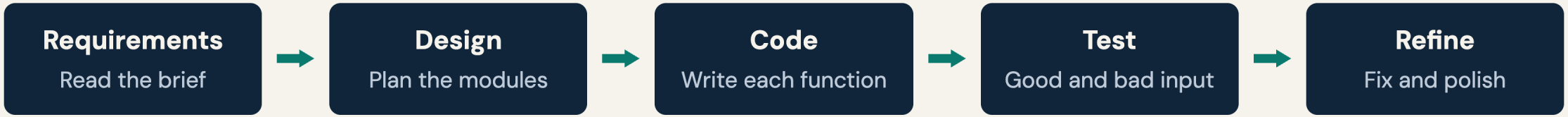
Safe input

Wrong amounts, impossible dates and future dates are rejected.

07 · PROJECT METHODOLOGY

Approach and technology stack

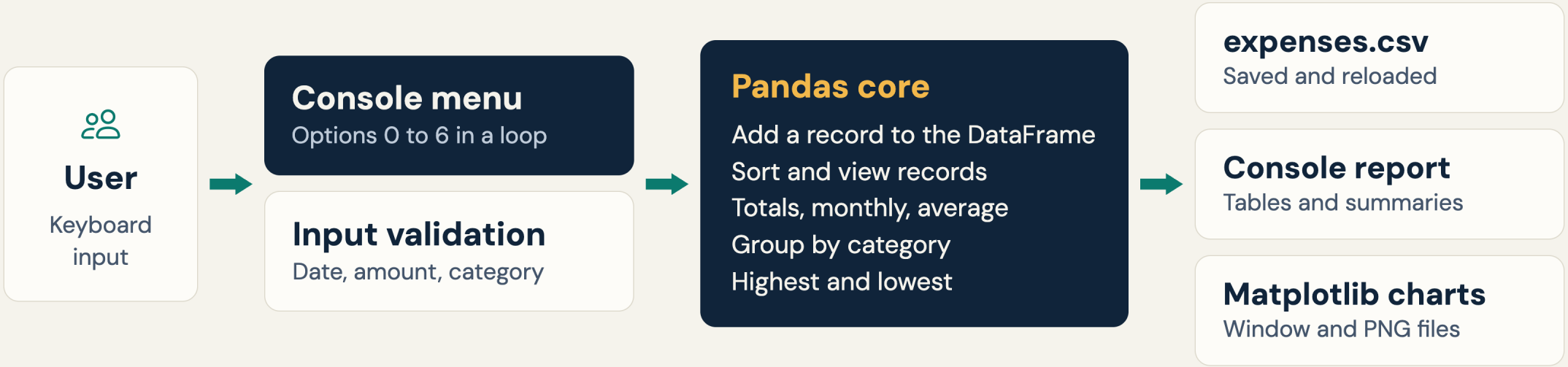
Development approach: iterative, one feature at a time



Technology stack

LANGUAGE **Python 3**	LIBRARIES **Pandas, Matplotlib**	DATABASE **CSV file (flat file)**
EDITOR **Visual Studio Code**	INTERFACE **Console text menu**	STRUCTURE **One function per feature**

System architecture and data flow



Data flow: input is validated, added to a Pandas DataFrame and saved to CSV; analysis then feeds the reports and charts. At start-up the CSV is loaded back into the DataFrame.

Module structure and data design

Modules in expense_tracker.py

Module	Functions
File I/O	load_expenses(), save_expenses()
Validation	ask_date(), ask_amount(), ask_category()
Records	add_expense(), view_expenses(), sort_expenses()
Analysis	summary(), monthly_report()
Charts	bar_chart(), pie_chart(), line_plot(), histogram()
Menu	main(): loop that calls the rest

Data design: expenses.csv

Field	Type	Example
Date	Date	2026-09-02
Category	Text	Education
Amount	Number > 0	4500.0
Description	Text, optional	Python course

Why a CSV file?

Plain text, opens in Excel, needs no database set-up and is easy to back up.

Key code: validation and analysis

Input validation: ask_amount()

```
def ask_amount():
    while True:
        text = input("Amount: ")
        try:
            amount = float(text.replace(",", ""))
        except ValueError:
            print("Invalid amount.")
            continue
        if not 0 < amount < math.inf:
            print("Must be positive.")
            continue
        return round(amount, 2)
```

Analysis with Pandas: summary()

```
# Monthly totals
monthly = df.groupby(
    pd.to_datetime(df["Date"]).dt.to_period("M")
)["Amount"].sum()

# Category-wise totals
by_cat = (df.groupby("Category")["Amount"]
          .sum().sort_values(ascending=False))

# Highest and lowest
hi = df.loc[df["Amount"].idxmax()]
lo = df.loc[df["Amount"].idxmin()]
```

Adding an expense with input validation

```
--- Add Expense ---
Date (YYYY-MM-DD or DD-MM-YYYY, Enter = today): 2026-08-03
Categories:
  1. Food
  2. Travel
  3. Education
  4. Shopping
  5. Entertainment
  6. Other
Choose category (number or name): 1
Amount: abc
  Invalid amount. Please enter a number, e.g. 250.50
Amount: -50
  Amount must be a positive number.
Amount: 250
Description (optional): Lunch at canteen
Saved: Food - Rs.250.00 on 2026-08-03
```

```
--- Add Expense ---
Date (YYYY-MM-DD or DD-MM-YYYY, Enter = today): 32-13-2026
  Invalid date. Example: 2026-10-07 or 07-10-2026
Date (YYYY-MM-DD or DD-MM-YYYY, Enter = today): 2099-01-01
  Date cannot be in the future.
Date (YYYY-MM-DD or DD-MM-YYYY, Enter = today): 2026-08-15
Categories:
  1. Food
  2. Travel
  3. Education
  4. Shopping
  5. Entertainment
  6. Other
Choose category (number or name): Travel
Amount: 1200
Description (optional): Train ticket
Saved: Travel - Rs.1,200.00 on 2026-08-15
```

Two real runs of option 1: invalid amounts and dates are rejected until valid input is entered.

Viewing, sorting and summarising records

Spending summary (option 4)

```
========== SPENDING SUMMARY ==========
Number of expenses   : 8
Total expenditure    : Rs.11,320.00
Average expenditure  : Rs.1,415.00 per expense

Monthly expenditure:
  2026-08  Rs.1,450.00
  2026-09  Rs.6,900.00
  2026-10  Rs.2,970.00
  (average per month: Rs.3,773.33)

Category-wise expenditure:
  Education       Rs.    4,500.00  ( 39.8%)
  Shopping        Rs.    4,000.00  ( 35.3%)
  Travel          Rs.    1,650.00  ( 14.6%)
  Entertainment   Rs.      600.00  (  5.3%)
  Food            Rs.      570.00  (  5.0%)

Highest expense:
  Rs.4,500.00 | Education | 2026-09-02 | Python course
Lowest expense:
  Rs.250.00 | Food | 2026-08-03 | Lunch at canteen
======================================
```

All records, then sorted (options 2 and 3)

```
--- All Expenses ---
      Date      Category   Amount      Description
2026-08-03          Food   250.00 Lunch at canteen
2026-08-15        Travel 1,200.00     Train ticket
2026-09-02     Education 4,500.00    Python course
2026-09-10      Shopping 1,800.00           Shoes
2026-09-21 Entertainment   600.00      Movie night
2026-10-01          Food   320.00           Dinner
2026-10-05        Travel   450.00         Bus pass
2026-10-12      Shopping 2,200.00       Headphones

Sort by:  1. Date   2. Amount   3. Category
Choice: 2
Ascending or descending? (a/d): d

--- Expenses sorted by Amount ---
      Date      Category   Amount      Description
2026-09-02     Education 4,500.00    Python course
2026-10-12      Shopping 2,200.00       Headphones
2026-09-10      Shopping 1,800.00           Shoes
2026-08-15        Travel 1,200.00     Train ticket
2026-09-21 Entertainment   600.00      Movie night
2026-10-05        Travel   450.00         Bus pass
2026-10-01          Food   320.00           Dinner
2026-08-03          Food   250.00 Lunch at canteen
```

Key figures

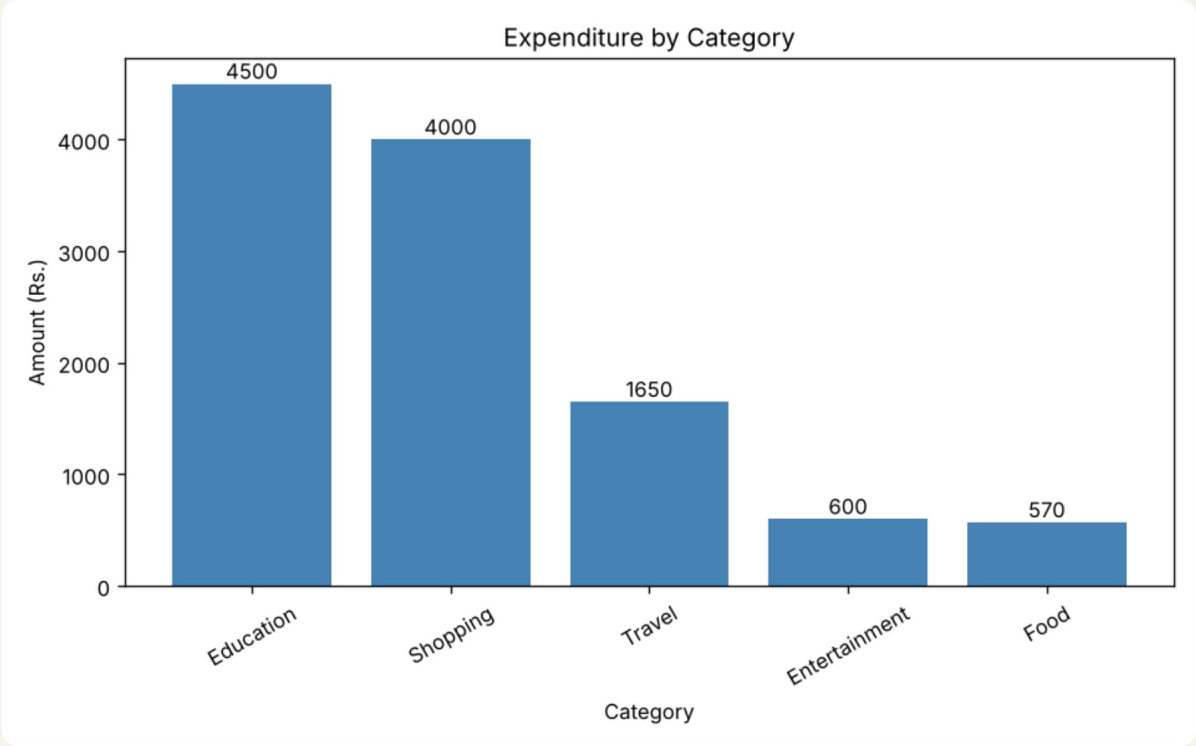
Rs.11,320
Total expenditure

Rs.1,415
Average per expense

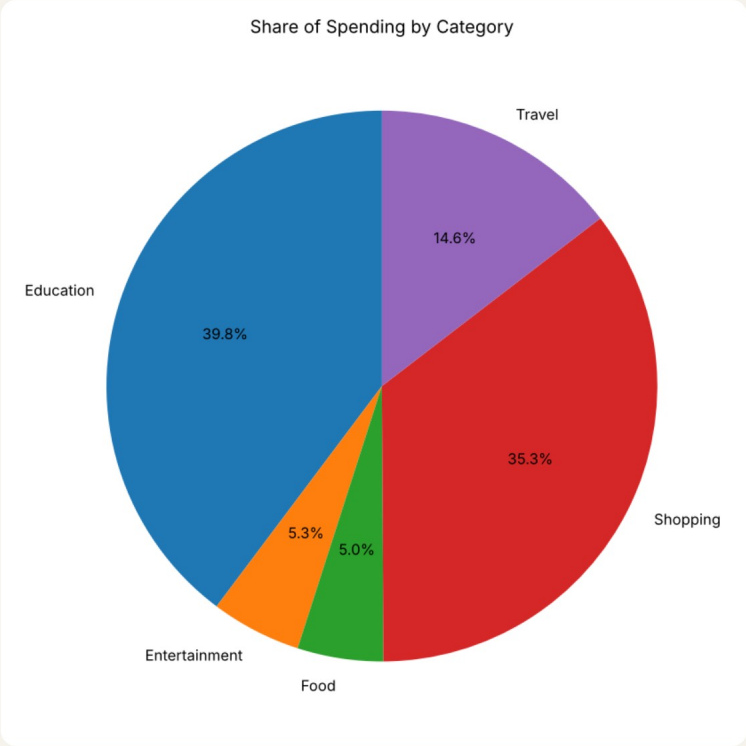
Rs.4,500
Highest: Education

Spending by category

Bar chart



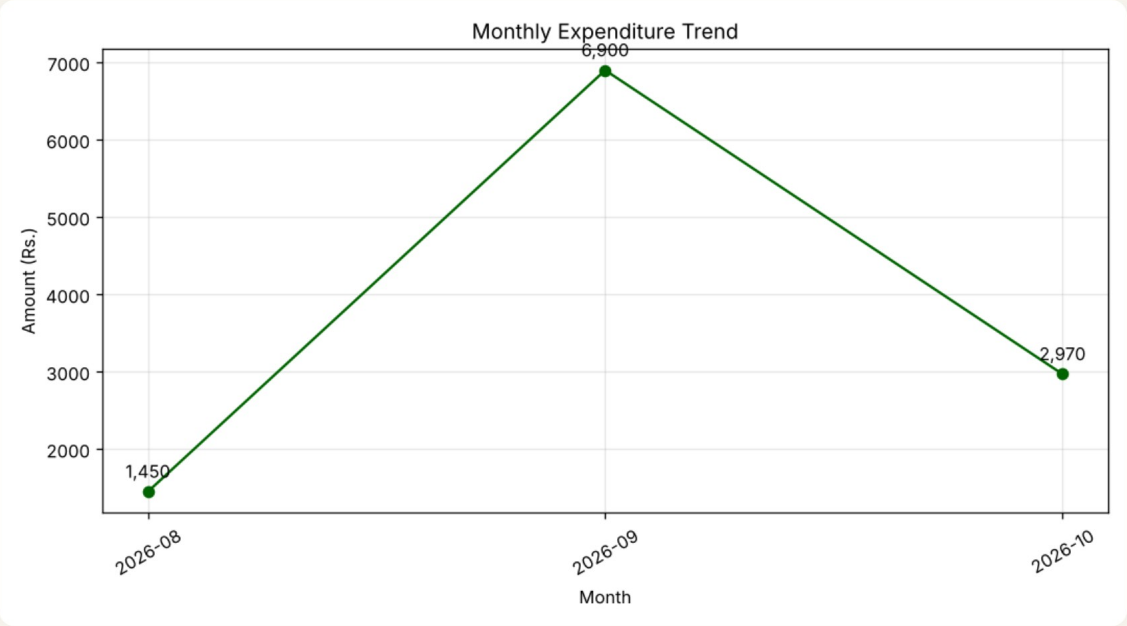
Pie chart



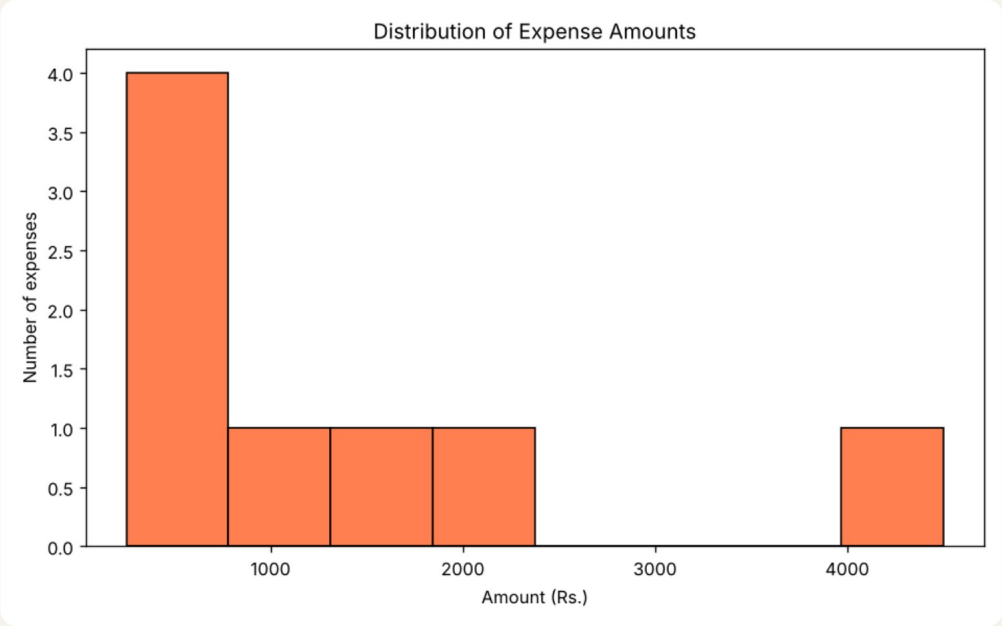
Education (Rs.4,500) and Shopping (Rs.4,000) together make up 75% of the spending.

Monthly trend and amount distribution

Line plot



Histogram



September was the costliest month (Rs.6,900); seven of eight expenses are below Rs.2,500.

Test cases and outcomes

#	Test case	Input	Expected result	Status
T1	Valid expense	2026-08-03, Food, 250	Saved to expenses.csv	Pass
T2	Text as amount	abc	Rejected, asks again	Pass
T3	Negative amount	-50	Rejected, asks again	Pass
T4	Impossible date	32-13-2026	Rejected, asks again	Pass
T5	Future date	2099-01-01	Rejected, asks again	Pass
T6	Category by name	shopping (lower case)	Accepted as Shopping	Pass
T7	Reload after restart	Run the program again	8 records loaded	Pass
T8	Total check	8 sample expenses	Rs.11,320, equals hand sum	Pass
T9	Wrong menu option	9	“Invalid option” message	Pass
T10	Chart generation	All four chart functions	4 PNG files created	Pass

Result: 10 of 10 test cases passed on a sample of 8 expenses.

11 · CONCLUSION

Objectives and how they were met

- Expenses saved with date, category, amount and description
- Records stored in a CSV file and reloaded on start-up
- Total, monthly, category-wise and average spending calculated
- Highest, lowest and sorted views available
- Bar, pie, line and histogram charts generated
- Invalid amounts and dates rejected

All six objectives met

The result is a working tool that turns daily entries into totals, summaries and charts, and it passed all 10 test cases.

12 · FUTURE SCOPE

Possible improvements and extensions

Edit and delete

Fix mistakes without opening the CSV file.

Budget alerts

Set a monthly limit per category and get a warning.

Graphical interface

A Tkinter or Streamlit front end in place of the text menu.

SQLite database

Faster queries and safer storage as data grows.

Export reports

Save summaries and charts as Excel or PDF files.

Spending forecast

Predict next month's spending from past trends.